

ラボ3.1: ネットワークキャプチャ分析

1. はじめに

1.1. 必要なリソース

- ControlThings プラットフォーム VM

1.2. まとめ

ラボでは、食品を包装している包装工場からのネットワークキャプチャを分析します。このキャプチャには4つの異なるICSプロトコルが含まれていますが、Modbus TCPに焦点を当てます。ラボ終了後に時間が残っていれば、キャプチャの分析を通じて、この会社と包装している製品についてさらに詳しく調べてみてください

1.3. 目的

- ICSプロトコル全般を分析する
- Modbus TCPを分析する
 - Modbusクライアントを識別する
 - Modbusサーバーを識別する
 - Modbus読み取り要求を識別する
 - Modbus書き込み要求を識別する
- プロトコルのベースラインを特定する

2. ラボの手順

ControlThings VMを起動する

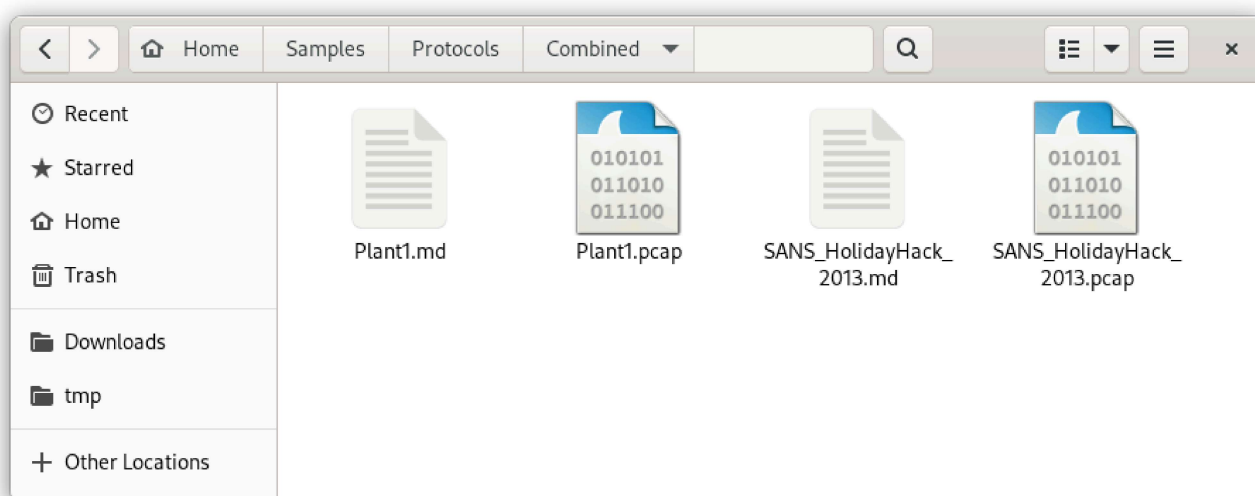
2.1. キャプチャを開く

~/Samples/Protocols/Combined/Plant1.pcap Wiresharkでファイルを開く

左上隅の「アクティビティ」メニューを開きます。画面下部に青いファイルキャビネットのアイコンが表示されます。



これはファイルアプリケーションです。クリックして開きます。ホームディレクトリに移動します。ホームディレクトリには、いくつかのフォルダがあります。「Samples」フォルダを開き、「Protocols」フォルダを開きます。そこには、含まれているプロトコルにちなんで名付けられたいくつかのフォルダがあります。「Combined」フォルダを開きます。このフォルダの中に「Plant1.pcap」というファイルがありますので、これを開いてください。Wiresharkで自動的に開くはずですが。



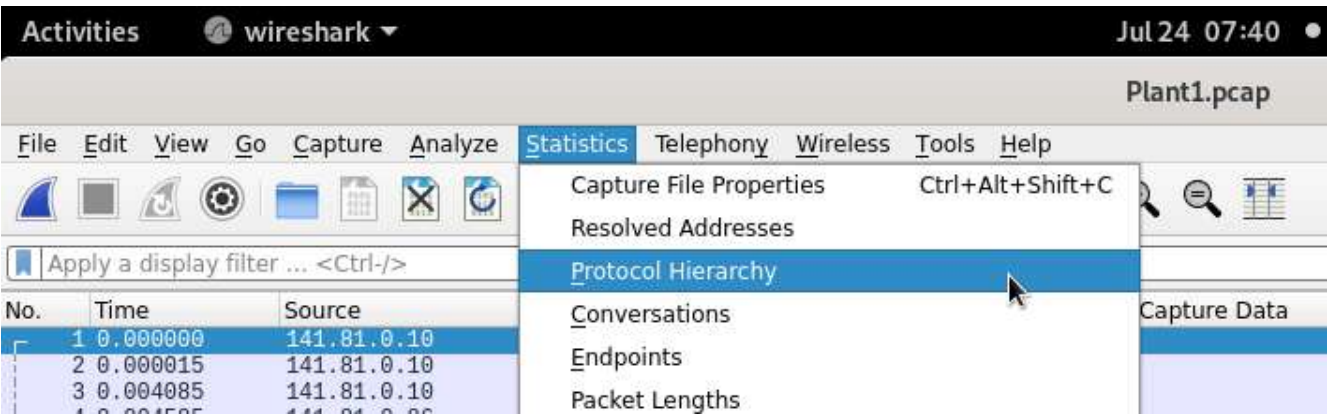
ヒント

前回のラボで作成したカスタムWireshark列を削除するには、列ヘッダーを右クリックし、「この列を削除」を選択します

2.2. プロトコル階層

「統計」メニューには、いくつかの便利なツールがあります。このラボでは、そのうちのいくつかを詳しく見ていきます。最初に紹介する「プロトコル階層」は、キャプチャに含まれるプロトコルと各プロトコルの割合を理解するのに役立ちます

Wiresharkメニューから、Statistics > Protocol Hierarchy



Activities wireshark Jul 24 07:40 Plant1.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No. Time Source

1 0.000000 141.81.0.10

2 0.000015 141.81.0.10

3 0.004085 141.81.0.10

4 0.004505 141.81.0.10

Capture File Properties Ctrl+Alt+Shift+C

Resolved Addresses

Protocol Hierarchy

Conversations

Endpoints

Packet Lengths

Capture Data

成功

Wireshark - Protocol Hierarchy Statistics - Plant1.pcap

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s
Frame	100.0	55800	100.0	6791668	639 k
Ethernet	100.0	55800	11.5	781200	73 k
PROFINET Real-Time Protocol	0.0	14	0.0	588	55
PROFINET DCP	0.0	14	0.0	298	28
Logical-Link Control	0.1	50	0.0	2784	262
Spanning Tree Protocol	0.1	42	0.0	1470	138
Dynamic Trunk Protocol	0.0	6	0.0	204	19
Cisco Discovery Protocol	0.0	2	0.0	920	86
Link Layer Discovery Protocol	0.0	17	0.1	3774	355
Internet Protocol Version 6	0.1	38	0.0	1520	143
User Datagram Protocol	0.0	24	0.0	192	18
Link-local Multicast Name Resolution	0.0	20	0.0	500	47
DHCPv6	0.0	4	0.0	362	34
Internet Control Message Protocol v6	0.0	14	0.0	436	41
Internet Protocol Version 4	99.6	55554	16.4	1111128	104 k
User Datagram Protocol	0.4	250	0.0	2000	188
Simple Service Discovery Protocol	0.0	2	0.0	266	25
NetBIOS Name Service	0.1	75	0.1	3804	358
Link-local Multicast Name Resolution	0.0	20	0.0	500	47
Distributed Computing Environment / Remote Procedure Call (DCE/RPC)	0.1	28	0.3	19376	1,824
Data	0.2	125	0.0	1780	167
Transmission Control Protocol	99.0	55249	70.5	4790714	451 k
Transport Layer Security	0.8	423	4.2	287752	27 k
TPKT - ISO on TCP - RFC1006	28.4	15852	0.9	63408	5,970
ISO 8073/X.224 COTP Connection-Oriented Transport Protocol	28.4	15852	0.7	47556	4,477
S7 Communication	18.9	10568	7.1	479343	45 k
Tabular Data Stream	2.4	1329	11.0	749224	70 k
Malformed Packet	0.4	220	0.0	0	0
Socks Protocol	0.0	8	0.0	832	78
NetBIOS Session Service	0.3	173	0.4	28657	2,698
SMB2 (Server Message Block Protocol version 2)	0.0	4	0.0	372	35

No display filter.

Close Copy Help

右側に、様々な測定値に分類された統計情報が表示されます。プロトコルについては、折りたたみ可能な階層構造になっているため、関心のある項目を簡単に表示/非表示にすることができます

? ICS プロトコルはいくつありますか?

- プロフィネット
- S7 Comm (シーメンスS7ファミリーコントローラー向け独自プロトコル)
- Modbus TCP
- EtherNet/IP-CIP

このキャプチャの75%以上は、Modbus、S7 Comm (シーメンスのS7ファミリーコントローラー用の独自プロトコル)、および EtherNet/IP-CIP による制御トラフィックで構成されています。ここではModbusトラフィックに焦点を当てますが、時間があれば、他のプロトコルについても詳しく調べてください

制御システムと IIoT の評価と活用コースでは、このキャプチャで Wireshark に解析機能がない ICS プロトコルをいくつか見つけました...

Protocol Hierarchy ウィンドウを閉じる

2.3. Modbusに焦点を当てる

このラボでは Modbus TCP プロトコルに焦点を当てているため、最初に Modbus TCP パケット キャプチャのみを表示するフィルターを構築します。

modbus Wiresharkのフィルターフィールドに入力する

Enter キーを押すのを忘れないでください...

✓ 成功

The screenshot shows the Wireshark interface with a filter applied to capture Modbus traffic. The packet list displays several Modbus/TCP packets, including queries and responses. The packet details pane for the selected packet (No. 38) shows the following structure:

- Frame 38: 80 bytes on wire (640 bits), 80 bytes captured (640 bits)
- Ethernet II, Src: Elau_02:58:b7 (00:04:17:02:58:b7), Dst: HewlettP_e0:02:5e (78:e7:d1:e0:02:5e)
- Internet Protocol Version 4, Src: 141.81.0.86, Dst: 141.81.0.10
- Transmission Control Protocol, Src Port: 502, Dst Port: 57184, Seq: 274, Ack: 25, Len: 26
- Modbus/TCP
 - Transaction Identifier: 0
 - Protocol Identifier: 0
 - Length: 7
 - Unit Identifier: 255

このフィルターは、Wiresharkに、TCPポート502に送信またはTCPポート502から送信されるTCPパケットのみを表示するように指示します。これは、Modbus TCPが通信に使用するポートです。Modbusサーバーは、Modbusクライアントからのコマンドをポート502でリッスンします。クライアントは、ランダムに選択されたTCPポート上のIPアドレスから、読み取りや書き込みなどのコマンドをModbusサーバーのIPアドレスのポート502に送信します。サーバーはこのコマンドを受信すると、TCPポート502上のIPアドレスから、クライアントのIPアドレスと、元のリクエストで使用されたのと同じランダムに選択されたTCPポートにパケットを送信して応答します。

パケットを調べて、この双方向通信の様子を確認してください。例えば、パケット5はIPアドレス141.81.0.10のTCPポート57184を使用して接続しているModbusクライアントからのリクエストで、141.81.0.86のModbusサーバーは標準ModbusポートであるTCP 502でリッスンしています。パケット6は、リクエストを受信したIPアドレスとポート番号と同じサーバーに情報を返すレスポンスです。

2.4. Modbus 会話

さて、**Modbus クライアントと Modbus サーバーはいくつあるか**という質問に答えましょう。すべてのパケットをスクロールして手動で数えることもできますが、時間がかかり、キャプチャのサイズが大きいため間違いが発生しやすくなります。最も簡単な方法は、Wireshark の「統計」メニューにある「会話」ツールを使用することです

Wiresharkメニューから、 **Statistics > Conversations**

IPv4 タブに移動

このキャプチャには、すべてのIPv4通信が表示されています。デフォルトでは、通信ウィンドウはメインウィンドウにある**Modbus**フィルターを使用していないことに注意してください。これは簡単に修正できます。

Limit to display filter ウィンドウの下部にある

✓ 成功

Wireshark · Conversations · Plant1

Ethernet · 13IPv4 · 13IPv6TCP · 14UDP

Address A	Address B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
141.81.0.10	141.81.0.86	1,077	100 k	555	40 k	522	59 k	0.004620	84.9284	3858	5561
141.81.0.10	141.81.0.24	1,131	92 k	544	37 k	587	55 k	0.077248	84.6366	3510	5217
141.81.0.10	141.81.0.44	1,005	81 k	480	33 k	525	48 k	0.087332	84.6117	3127	4552
141.81.0.10	141.81.0.104	1,032	82 k	495	33 k	537	48 k	0.087466	84.8753	3198	4598
141.81.0.10	141.81.0.144	784	66 k	371	25 k	413	40 k	0.087562	84.8752	2426	3850
141.81.0.10	141.81.0.164	787	66 k	372	25 k	415	40 k	0.087778	84.8753	2433	3862
141.81.0.10	141.81.0.26	715	62 k	363	26 k	352	35 k	0.132325	84.5331	2507	3382
141.81.0.10	141.81.0.66	1,056	98 k	535	39 k	521	58 k	0.178421	84.6450	3770	5573
141.81.0.10	141.81.0.46	559	51 k	285	21 k	274	30 k	0.184431	84.5310	2019	2901
141.81.0.10	141.81.0.64	1,063	89 k	510	34 k	553	54 k	0.264695	84.4253	3310	5169
141.81.0.10	141.81.0.84	1,103	87 k	530	36 k	573	51 k	0.300681	84.4264	3434	4841
141.81.0.10	141.81.0.143	739	74 k	382	29 k	357	45 k	0.357634	84.5383	2762	4322
141.81.0.10	141.81.0.163	830	79 k	426	31 k	404	47 k	0.434385	84.4844	2989	4528

これで、Modbus TCPトラフィックのみを確認できるようになりました。

このビューにより、このネットワーク内にあるModbusクライアントとサーバーの数を簡単に確認できます。クライアントとサーバー間の会話ごとに1行あります。アドレスAのデバイスは会話を初期化したデバイスで、アドレスBのデバイスは応答を行っているデバイスです。ここでは、すべてのインスタンスでIPアドレス141.81.0.10が会話を初期化していることがわかります。Modbus TCPプロトコルにはサーバーが独自の会話を開始できるインスタンスがないため、これはModbusクライアントである必要があります。アドレスA列に他のIPアドレスが表示されないため、このネットワークにはModbusクライアントが1つだけあるはずです。

サーバーに関しては、13 種類の異なる会話があることがわかります。つまり、Modbus クライアントが通信しているサーバーは13 種類あるはずです。

インシデント対応やセキュリティ評価を行う際に、このウィンドウはネットワーク内の通常のトラフィックのベースラインを構築するのに非常に役立ち、Modbus トラフィックに異常が発生しているかどうかをいつでも判断するのに役立ちます。

2.5. Modbus機能によるフィルタリング

141.81.0.10 がサーバーのプロセスを変更または制御するコマンドを送信しているかどうかを確認したい場合は、書き込みコマンドを検索してください。Modbus には 4 つのデータ書き込みコマンドがあります。

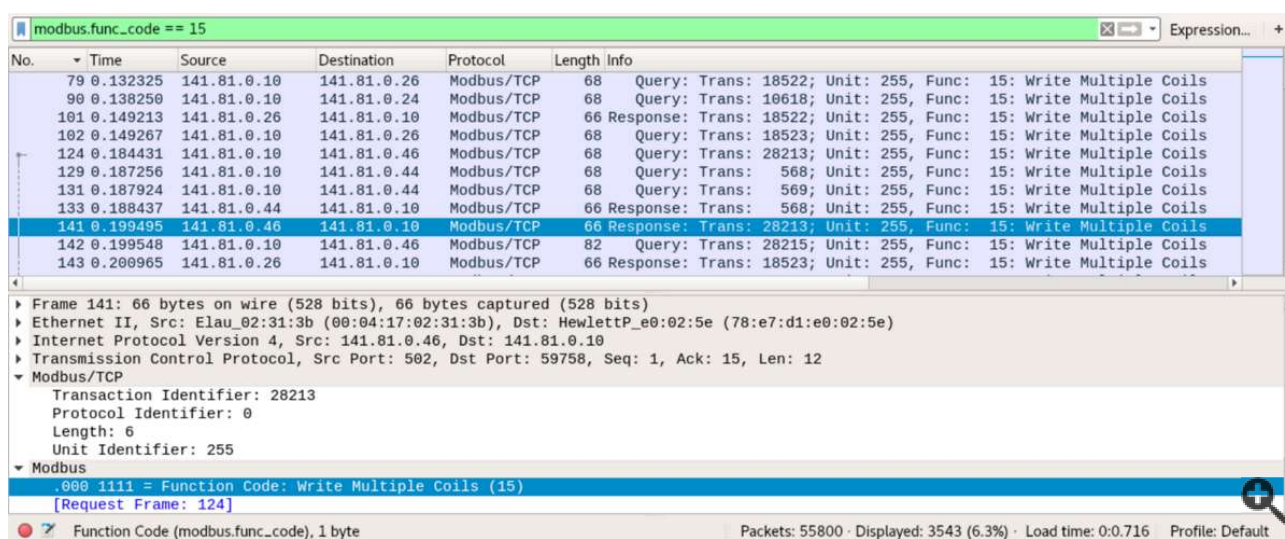
5 - Write Single Coil
6 - Write Single Register
15 - Write Multiple Coils
16 - Write Multiple Registers

Wiresharkの変数を使えば、これらのコマンドに基づいてフィルタリングできます `modbus.func_code`。このフィルタ文字列がわからない場合は、Wireshark **Display Filter Expression** ツールのModbusフィルタを調べて確認することができます。フィルタフィールドを右クリックするか、Wiresharkの **Analyze** メニューから選択してください。また、Modbusパケットを見つけて、関数コード行を右クリックし、「フィルタとして適用」を選択すると、適切なフィルタが自動生成されます。その後、ニーズに合わせてフィルタを変更できます。

Wiresharkフィルタを変更する `modbus.func_code == 15`

必ず2つの等号を使用し、最後にEnterキーを押します。そうしないと機能しません。

✓ 成功



上記の4つの書き込みコマンドがそれぞれ存在するかどうかを確認します。

関数5と6に一致するパケットは見つかりません。しかし、コマンド15と16を実行すると、ModbusクライアントがModbusサーバーのレジスタに書き込みを行っているパケットがいくつか生成されるはずです。つまり、クライアントがコイルやレジスタを変更することでサーバーを積極的に制御しているという証拠が得られました。

2.6. トラフィックのベースライン

Wiresharkの統計メニューにあるIOグラフ機能を使用すると、キャプチャの真ん中、42秒のタイムスタンプ付近にトラフィックの小さなスパイクがあることがわかります

フィルターバーからフィルターを削除します

Wiresharkメニューから、Statistics > I/O Graph

パケットではなくバイト単位でグラフ化すると、スパイクがさらに大きくなることがわかります。

!!! 例 "表のY軸の列の下で、パケットをダブルクリックして、両方の行をバイトに変更します。

Modbus TCPがトラフィックの急増に関係していたかどうかを判断したい場合は、Wiresharkのフィルターを使ってグラフ下の表に線を追加し、グラフに表示する項目を選択できます。表の最初の行は、グラフの黒い線で、1秒ごとのポイントで総トラフィック量を表示します。2行目は、よく見るとほとんど見えないエラーの棒グラフです。3行目を追加して、Modbusトラフィックを表示することもできます。

左下の+ボタンをクリックして、表に新しい行を追加します。

3行目のグラフ名フィールドをダブルクリックして「すべてのパケット」から「Modbus*」に変更します。

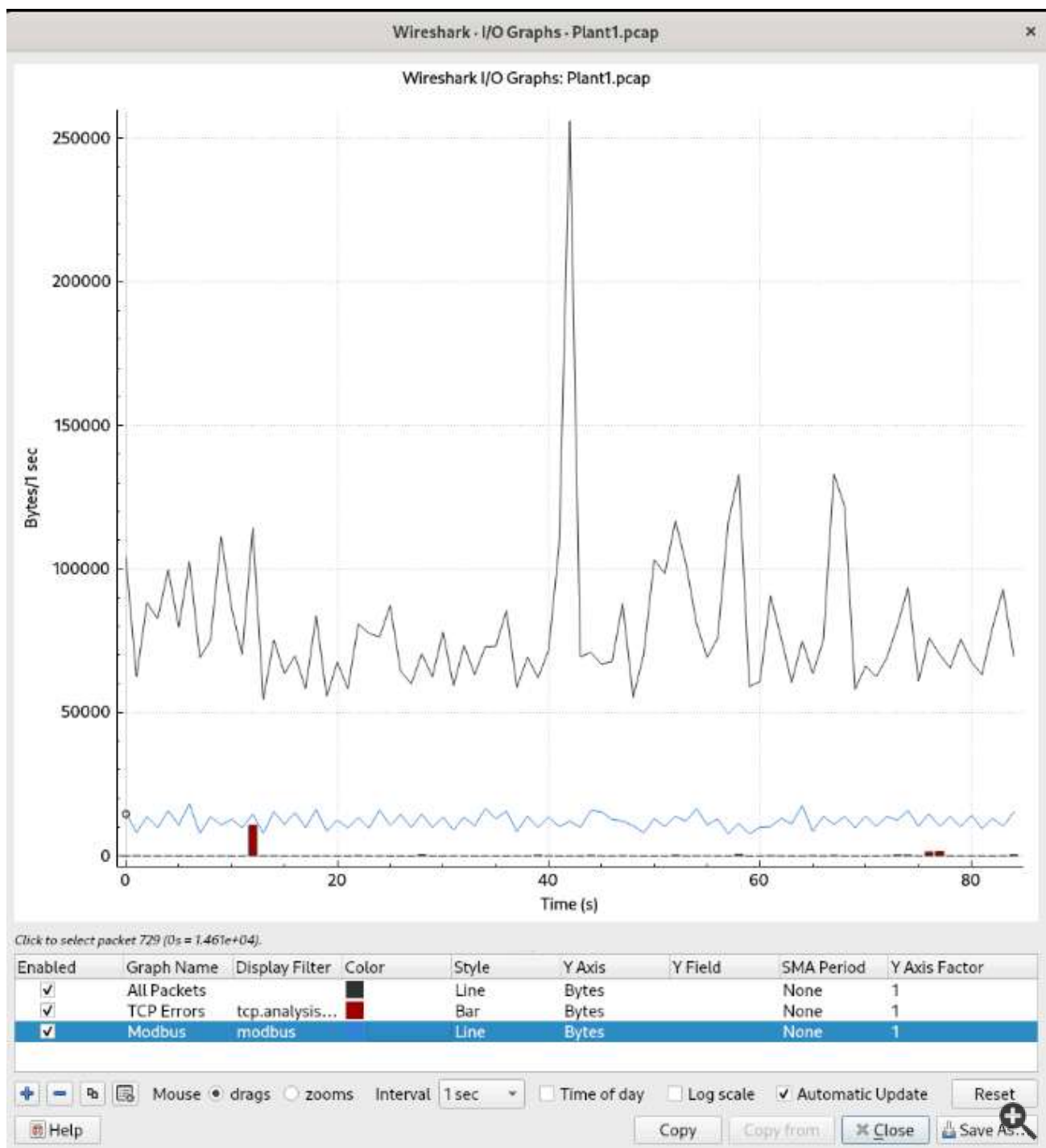
表示フィルターフィールドをModbusに変更します

色フィールドを変更し、ミディアムブルーを選択します

Y軸フィールドをバイトに変更します

左端の「有効」ボックスをチェックしてModbusラインを有効にします。

✓ 成功



誤ってグラフを拡大してしまい、元に戻らない場合は、右下のリセットボタンを使用してください

このグラフでは、Modbus TCP トラフィックがこのキャプチャ内のネットワークトラフィック全体の約 4 分の 1 を占めており、その通信速度は一定であることがわかります。

グラフに線を追加して、他の種類のトラフィックのグラフ化を少し試してみましょう。例えば、フィルターを使って、4つの制御プロトコルの合計を線でグラフ化できます `enip || modbus || pn_rt || s7comm`。これらがすべて平坦で一貫性があることに注目してください。これはほとんどのICSで共通しています。これは、制御トラフィックがトラフィックの急増の原因ではないことを示しています。

3. ボーナスチャレンジ

他の参加者のラボの完了を待っている場合、またはこのラボで紹介された概念をさらに深く探求したい場合は、以下のアイデアを試してみてください。これらのチャレンジにはチュートリアルは用意されていませんが、行き詰まった場合は、コース作成者のジャスティンに取り組みの成果を示せば、喜んでヒントを教えてください。justin [@controlthings.io](mailto:justin@controlthings.io)までメールでお問い合わせください

3.1. 急増の原因は何ですか?

この急上昇の原因となっているプロトコルを特定してみましょう。ヒントとして、Wiresharkの「統計」>「エンドポイント」機能を使用し、「TCP」タブでどのプロトコルが最も多くのパケットとバイトを送信しているかを確認することが挙げられます。どのプロトコルが最も多くの通信を行っているかを確認したら、「IOグラフ」ウィンドウに戻り、それらのプロトコルをグラフ化してみましょう。別の方法として、IOグラフを拡大表示し、急上昇部分をクリックすると、Wiresharkがメインウィンドウのその時間帯までスクロールします。この方法では、正確な情報を得ることができませんが、実際のパケットやプロトコルを特定することはできません。しかし、グラフ化を試みて、一致するグラフを見つけられるかどうかを確認できるヒントは得られます。